

React Interview Questions and Answers

React is an efficient, flexible, and open-source **JavaScript library** that allows developers to create simple, fast, and scalable web applications. Jordan Walke, a software engineer who was working for Facebook, created React. Developers with a JavaScript background can easily develop web applications with React.

1. What is ReactJS?

[ReactJS](#) is a JavaScript library used to build reusable components for the view layer in the MVC architecture. It is used to build the Single Page Application (SPA) due to its component-based architecture, efficient re-rendering with the Virtual DOM, and ability to manage dynamic content without needing full page reloads. It is written in JSX.

Important Features of React:

- **Virtual DOM:** React uses a virtual DOM to efficiently update and render components, ensuring fast performance by minimizing direct DOM manipulations.
- **Component-Based Architecture:** React builds UI using reusable, isolated components, making code more modular, maintainable, and scalable.
- **Hooks:** React Hooks allow functional components to manage state and side effects, making them powerful and more flexible.
- **Server-Side Rendering (SSR):** React can be used for server-side rendering, where HTML content is generated on the server and sent to the client. This improves the app's performance, especially for SEO.
- **React Router:** React Router enables navigation in a React application. It allows you to define different routes for different views in a single-page application (SPA).

2. What is the latest version of the React?

The latest stable version of React is **v19.1.0**, released on **March 28, 2025**. This version builds upon the major updates introduced in **React v19.0.0**, which was officially released on **December 5, 2024**.

3. Explain the MVC architecture.

The [Model-View-Controller \(MVC\)](#) framework is an architectural/design pattern that separates an application into three main logical components: Model, View, and Controller. Each architectural component is built to handle specific development aspects of an application. It isolates the business, logic, and presentation layers from each other

4. Explain the building blocks of React.

The five main building blocks of React are

- **Components:** These are reusable blocks of code that return [HTML](#).
- **JSX:** It stands for JavaScript and XML and allows you to write HTML in [React](#).
- **Props and State:** props are like function parameters and State is similar to variables.
- **Context:** This allows data to be passed through components as props in a hierarchy.
- **Virtual DOM:** It is a lightweight copy of the actual DOM which makes DOM manipulation easier.

5. Explain props and state in React with differences

Props are used to pass data from one component to another. The state is local data storage that is local to the component only and cannot be passed to other components.

PROPS	STATE
The Data is passed from one component to another.	The Data is passed within the component only.
It is Immutable (cannot be modified).	It is Mutable (can be modified).
Props can be used with state and functional components.	The state can be used only with the state components/class component (Before 16.0).

PROPS	STATE
Props are read-only.	The state is both read and write.

6. What is virtual DOM in React?

The **Virtual DOM** in React is an in-memory representation of the actual DOM. It helps React efficiently update and render the user interface by comparing the current and previous virtual DOM states using a process called **diffing**.

How Virtual DOM Works

- **Efficient Rendering:** The Virtual DOM is an in-memory representation of the actual DOM that React uses to optimize the process of updating and rendering UI changes.
- **Diffing Algorithm:** React compares the current and previous versions of the Virtual DOM using a diffing algorithm, identifying the minimal set of changes required to update the real DOM.
- **Batch Updates:** Instead of updating the real DOM immediately, React batches multiple changes to reduce unnecessary re-renders, improving performance.
- **Faster Updates:** Since updating the real DOM is slow, React minimizes direct DOM manipulations by only making updates where necessary after comparing the Virtual DOM.
- **Declarative UI:** With the Virtual DOM, React allows developers to write code in a declarative style, letting React handle when and how to efficiently update the UI.

7. Differentiate between Real DOM and virtual DOM?

Real DOM	Virtual DOM
The actual DOM, a tree-like structure representing the UI elements.	A lightweight copy of the Real DOM used to optimize updates.

Real DOM	Virtual DOM
Slower as it requires direct updates to the actual DOM.	Faster because it minimizes direct manipulation of the Real DOM.
Directly manipulates the Real DOM, causing re-rendering of the entire UI.	Updates are made to the Virtual DOM first, then changes are batched and only the necessary changes are reflected in the Real DOM.
Entire UI might need to be re-rendered when changes occur.	Only the necessary components are re-rendered, reducing unnecessary re-renders.
Less efficient due to repeated direct updates to the Real DOM.	More efficient by minimizing direct DOM manipulation and batch updates.

8. What is JSX?

[JSX](#) is basically a syntax extension of regular JavaScript and is used to create React elements. These elements are then rendered to the React DOM. All the React components are written in JSX. To embed any JavaScript expression in a piece of code written in JSX we will have to wrap that expression in curly braces {}.

Example of JSX: The name written in curly braces { } signifies JSX

```
const name = "Learner";
const element = (
  <h1>
    Hello,
    {name}.Welcome to GeeksforGeeks.
  </h1>
);
```

9. What are components and their type in React?

A [Component](#) is one of the core building blocks of React. In other words, we can say that every application you will develop in React will be made up of pieces called components. Components make the task of building UIs much easier.

In React, we mainly have two types of components:

- **Functional Components:** Functional components are simply JavaScript functions. Initially, they were limited in terms of features like state and lifecycle methods. However, with the introduction of Hooks, functional components can now use state, manage side effects, and access other features that were once exclusive to class components.
- **Class Components:** Class components are more complex than functional components. They are able to manage state, handle lifecycle methods, and can also interact with other components. Class components can pass data between each other via props, similar to functional components.

10. How do browsers read JSX?

In general, browsers are not capable of reading [JSX](#) and only can read pure JavaScript. The web browsers read JSX with the help of a transpiler. Transpilers are used to convert JSX into JavaScript. The transpiler used is called [Babel](#).

11. Explain the steps to create a react application and print Hello World?

To install React, first, make sure Node is installed on your computer. After installing Node. Open the terminal and type the following command.

```
npx create-react-app <<Application_Name>>
```

Note: The above method is now deprecated, so use the below given method for creating the React application.

```
npm create vite@latest
```

Navigate to the folder.

```
cd <<Application_Name>>
```

This is the first code of ReactJS Hello World!

```
import React from "react";
```

```
import "./App.css";
```

```
function App() {  
  return <div className="App">Hello World !</div>;  
}  
  
export default App;
```

Output:

Type the following command to run the application

npm start

Hello World !

12. How to create an event in React?

To create an [event](#) in React, attach an event handler like `onClick`, `onChange`, etc., to a [JSX](#) element. Define the handler function to specify the action when the event is triggered, such as updating state or executing logic.

```
function Component() {  
  doSomething(e);  
  
  {  
    e.preventDefault();  
  
    // Some more response to the event  
  }  
  
  return <button onClick={doSomething}></button>;  
}
```

13. Explain the creation of a List in React?

[Lists](#) are very useful when it comes to developing the UI of any website. Lists are mainly used for displaying menus on a website. To create a list in React use the `map` method of array as follows.

```
import React from "react";  
  
import ReactDOM from "react-dom/client"; // Import react-dom/client
```

```
const numbers = [1, 2, 3, 4, 5];

const updatedNums = numbers.map((number) => {
  return <li key={number}>{number}</li>; // Add a unique "key" prop
});

const root = ReactDOM.createRoot(document.getElementById("root")); //
Create the root

root.render(<ul>{updatedNums}</ul>); // Render the list into the root element
```

Output:

- 1
- 2
- 3
- 4
- 5

14. What is a key in React?

A [key](#) is a special string attribute you need to include when creating lists of elements in React. Keys are used in React to identify which items in the list are changed, updated, or deleted. In other words, we can say that keys are used to give an identity to the elements in the lists.

15. How to write a comment in React?

There are two ways to write [comments](#) in React.

- **Multi-line comment:** We can write multi-line comments in React using the asterisk format `/* */`.

```
/*
```

This is a multi-line comment.

It can span multiple lines.

```
*/
```

- **Single line comment:** We can write single comments in React using the double forward slash `//`.

// This is a single-line comment

16. Explain the difference between React and Angular?

React	Angular
React is a JavaScript library. As it indicates react js updates only the virtual DOM is present and the data flow is always in a single direction.	Angular is a framework. Angular updates the Real DOM and the data flow is ensured in the architecture in both directions.
React is more simplified as it follows MVC ie., Model View Control.	The architecture is complex as it follows MVVM models ie., Model View-ViewModel.
It is highly scalable.	It is less scalable than React JS.
It supports Uni-directional data binding which is one-way data binding.	It supports Bi-directional data binding which is two way data binding.
It has a virtual DOM.	It has regular DOM.

17. Explain the use of render method in React?

[React renders](#) HTML to the web page by using a function called render(). The purpose of the function is to display the specified HTML code inside the specified HTML element. In the render() method, we can read props and state and return our JSX code to the root component of our app.

18. What is state in React?

The [state](#) is an instance of React Component Class that can be defined as an object of a set of observable properties that control the behaviour of the component. In other words, the State of a component is an object that holds some information that may change over the lifetime of the component.

19. Explain props in React?

React allows us to pass information to a Component using something called props (which stands for properties). [Props](#) are objects which can be used inside a component

We can access any props inside from the component's class to which the props is passed. The props can be accessed as shown below:

```
this.props.propName;
```

20. What is higher-order component(HOC) in React?

[Higher-order components](#) or HOC is the advanced method of reusing the component functionality logic. It simply takes the original component and returns the enhanced component. HOC are beneficial as they are easy to code and read. Also, helps to get rid of copying the same logic in every component.

21. Explain the difference between functional and class component in React?

Functional Components	Class Components
A functional component is just a plain JavaScript pure function that accepts props as an argument	A class component requires you to extend from React. Component and create a render function
No render method used	It must have the render() method returning JSX
Also known as Stateless components	Also known as Stateful components
React lifecycle methods (for example, componentDidMount) cannot be used in functional components.	React lifecycle methods can be used inside class components (for example, componentDidMount).

Functional Components	Class Components
Constructors are not used.	Constructor is used as it needs to store state.
Uses hooks like useState for managing state.	Uses this.state and this.setState for state management.

22. Explain one way data binding in React?

ReactJS uses one-way [data binding](#) which can be Component to View or View to Component. It is also known as one-way data flow, which means the data has one, and only one way to be transferred to other parts of the application. In essence, this means child components are not able to update the data that is coming from the parent component. It is easy to debug and less prone to errors.

23. What is Context API in React?

The [Context API](#) is a way to share data (such as theme, language preference, etc.) across components without having to pass props down manually at every level. It provides a Provider component to set the value and a Consumer component or useContext() hook to access it.

24. What is the role of shouldComponentUpdate() in React?

[shouldComponentUpdate\(\)](#) is a lifecycle method that allows you to control whether a component should re-render when it receives new props or state. If it returns false, React will skip the re-render process for that component.

25. What is the use of dangerouslySetInnerHTML in React?

[dangerouslySetInnerHTML](#) is an attribute used to set raw HTML inside a component. It is generally discouraged because it can expose the application to XSS (cross-site scripting) attacks, but it is sometimes used for rendering third-party HTML content.

26. What are Pure Components in React?

A [Pure Component](#) is a type of React component that only re-renders if the props or state it receives change. React provides React.PureComponent, which

is a base class that automatically performs a shallow comparison of props and state to determine if a re-render is necessary.

27. What is the significance of `setState()` in React?

[`setState\(\)`](#) is a method used to update the state of a component. When the state is updated, React re-renders the component and its child components to reflect the changes.

28. What is conditional rendering in React?

[Conditional rendering](#) in React involves selectively rendering components based on specified conditions. By evaluating these conditions, developers can control which components are displayed, allowing for dynamic and responsive user interfaces in React applications.

Let us look at this sample code to understand conditional rendering.

```
{isLoggedIn === false ? <DisplayLoggedOut /> : <DisplayLoggedIn />}
```

Here if the boolean `isLoggedIn` is false then the `DisplayLoggedOut` component will be rendered otherwise `DisplayLoggedIn` component will be rendered.

29. What is React Router?

[React Router](#) is a standard library for routing in React. It enables the navigation among views of various components in a React Application, allows changing the browser URL, and keeps the UI in sync with the URL.

To install react router type the following command.

```
npm i react-router-dom
```

30. Explain the components of a react-router.

The main [components of a react-router](#) are:

1. **Router(usually imported as `BrowserRouter`):** It is the parent component that is used to store all of the other components. Everything within this will be part of the routing functionality
2. **Switch:** The switch component is used to render only the first route that matches the location rather than rendering all matching routes.

3. **Route:** This component checks the current URL and displays the component associated with that exact path. All routes are placed within the switch components.
4. **Link:** The Link component is used to create links to different routes.

31. How is React routing different from conventional routing?

React Routing	Conventional Routing
Used in Single Page Applications (SPA).	Typically used in Multi-Page Applications (MPA).
No full page reloads. React updates only the necessary parts of the UI.	Full page reload is triggered for every navigation request.
React uses a client-side router (e.g., React Router) to handle navigation and manage different views within the same page.	Uses server-side routing where the server responds with new HTML for each navigation.
Routes are managed by JavaScript, and the browser's address bar reflects the application's state.	Routes correspond to different server-side routes, and the server responds with new HTML files.

32. Explain the lifecycle methods of components

A React Component can go through [four stages](#) of its life as follows.

- **Initialization:** This is the stage where the component is constructed with the given Props and default state. This is done in the constructor of a Component Class.
- **Mounting:** Mounting is the stage of rendering the JSX returned by the render method itself.
- **Updating:** Updating is the stage when the state of a component is updated and the application is repainted.

- **Unmounting:** As the name suggests Unmounting is the final step of the component lifecycle where the component is removed from the page.

33. Explain the methods used in mounting phase of components

[Mounting](#) is the phase of the component lifecycle when the initialization of the component is completed and the component is mounted on the DOM and rendered for the first time on the webpage. The mounting phase consists of two such predefined functions as described below

- [componentWillMount\(\)](#) Function: This function is invoked right before the component is mounted on the DOM.
- [componentDidMount\(\)](#) Function: This function is invoked right after the component is mounted on the DOM.

34. What is this.setState function in React?

We use the [setState\(\)](#) method to change the state object. It ensures that the component has been updated and calls for re-rendering of the component. The state object of a component may contain multiple attributes and React allows using `setState()` function to update only a subset of those attributes as well as using multiple `setState()` methods to update each attribute value independently.

35. What is the use of ref in React?

[Refs](#) are a function provided by React to access the DOM element and the React element that you might have created on your own. They are used in cases where we want to change the value of a child component, without making use of props and all. They have wide functionality as we can use callbacks with them.

The syntax to use ref is :

```
const node = this.myCallRef.current;
```

36. What are hooks in React?

[Hooks](#) are a new addition in React 16.8. They let developers use state and other React features without writing a class. Hooks doesn't violate any existing React concepts. Instead, Hooks provide a direct API to react concepts such as props, state, context, refs and life-cycle

37. Explain the useState hook in React?

The most used hook in React is the [useState\(\)](#) hook. Using this hook we can declare a state variable inside a function but only one state variable can be declared using a single useState() hook. Whenever the useState() hook is used, the value of the state variable is changed and the new variable is stored in a new cell in the stack.

Syntax:

```
const [state, setState] = useState(initialState);
```

- **state:** The current state value.
- **setState:** A function used to update the state value.
- **initialState:** The initial value of the state.

38. Explain the useEffect hook in React?

The [useEffect](#) hook in React eliminates the side effect of using class based components. It is used as an alternative to componentDidMount() method. The useEffect hook accepts two arguments where second argument is optional.

```
useEffect(function, dependency)
```

The dependency decides when the component will be updated again after rendering.

39. What is React Fragments?

when we are trying to render more than one root element we have to put the entire content inside the 'div' tag which is not loved by many developers. So since React 16.2 version, [Fragments](#) were introduced, and we use them instead of the extraneous 'div' tag. The following syntax is used to create fragment in react.

```
<React.Fragment>
  <h2>Child-1</h2>
  <p> Child-2</p>
</React.Fragment>
```

40. What is a react developer tool?

[React Developer Tools](#) is a Chrome DevTools extension for the React JavaScript library. A very useful tool, if you are working on React applications. This extension adds React debugging tools to the Chrome Developer Tools. It

helps you to inspect and edit the React component tree that builds the page, and for each component, one can check the props, the state, hooks, etc.

41. How to use styles in ReactJS?

CSS modules are a way to locally scope the content of your CSS file. We can create a CSS module file by naming our CSS file as App.modules.css and then it can be imported inside App.js file using the special syntax mentioned below.

Syntax:

```
import styles from './App.module.css';
```

42. Explain styled components in React?

[Styled-component](#) Module allows us to write CSS within JavaScript in a very modular and reusable way in React. Instead of having one global CSS file for a React project, we can use styled-component for enhancing the developer experience. It also removes the mapping between components and styles – using components as a low-level styling construct

The command to install styled components is

```
npm i styled-components
```

Using the below code we can custom style a button in React

```
import styled from 'styled-components'
```

```
const Button = styled.div`
```

```
  width : 100px ;
```

```
  cursor: pointer ;
```

```
  text-decoration : none;
```

```
export default Button;
```

43. What is prop drilling and its disadvantages?

[Prop drilling](#) is basically a situation when the same data is being sent at almost every level due to requirements in the final level. The problem with Prop Drilling is that whenever data from the Parent component will be needed, it would have to come from each level, Regardless of the fact that it is not needed there and simply needed in last.

44. What is conditional rendering in React?

[Conditional rendering](#) in React is used when you want to render different UI elements based on certain conditions. For example, rendering a login button if a user is not logged in or rendering a logout button when the user is logged in.

```
const isLoggedIn = true;

return (
  <div>
    {isLoggedIn ? <button>Logout</button> : <button>Login</button>}
  </div>
);
```

45. What are controlled components in React?

[Controlled components](#) are React components where the form data is handled by the state within the component. The form element's value is controlled via the React state, making the form elements responsive to changes.

46. What is custom hooks in React?

[Custom hooks](#) are normal JavaScript functions whose names start with “use” and they may call other hooks. We use custom hooks to maintain the DRY concept that is Don't Repeat Yourself. It helps us to write a logic once and use it anywhere in the code.

47. How to optimize a React code?

We can improve our react code by following these practices:

- Using binding functions in constructors
- Eliminating the use of inline attributes as they slow the process of loading
- Avoiding extra tags by using React fragments
- Lazy loading

48. What is the difference between useRef and createRef in React ?

Here is the difference table of useRef and createRef in React

useRef	createRef
It is a hook.	It is a function.
It uses the same ref throughout the component's lifecycle.	It creates a new ref every time component re-renders.
It saves its value between re-renders in a functional component.	It creates a new ref for every re-render.
It returns a mutable ref object (i.e., can be changed).	It returns a read-only ref object (cannot be modified directly).
Used for accessing DOM elements, persisting values, or managing timers in functional components.	Used for class components, especially when referencing DOM elements.
<code>const myRef = useRef();</code>	<code>const myRef = React.createRef();</code>

49. What is react-redux?

[React-redux](#) is a state management tool which makes it easier to pass these states from one component to another irrespective of their position in the component tree and hence prevents the complexity of the application. As the number of components in our application increases it becomes difficult to pass state as props to multiple components. To overcome this situation we use react-redux

50. What are benefits of using react-redux?

They are several [benefits](#) of using react-redux such as:

- It provides centralized state management i.e. a single store for whole application
- It optimizes performance as it prevents re-rendering of component

- Makes the process of debugging easier
- Since it offers persistent state management therefore storing data for long times become easier

51. Explain the core components of react-redux?

There are four fundamental concepts of redux in react which decide how the data will flow through components

- **Redux Store:** It is an object that holds the application state
- **Action Creators:** These are functions that return actions (objects).
- **Actions:** Actions are simple objects which conventionally have two properties- type and payload
- **Reducers:** Reducers are pure functions that update the state of the application in response to actions

52. How can we combine multiple reducers in React?

When working with [Redux](#) we sometimes require multiple reducers. In many cases, multiple actions are needed, resulting in the requirement of multiple reducers. However, this can become problematic when creating the Redux store. To manage the multiple reducers we have function called `combineReducers` in the redux. This basically helps to combine multiple reducers into a single unit and use them.

Syntax

```
import { combineReducers } from "redux";
const rootReducer = combineReducers({
  books: BooksReducer,
  activeBook: ActiveBook
});
```

53. Explain CORS in React?

In ReactJS, [Cross-Origin Resource Sharing \(CORS\)](#) refers to the method that allows you to make requests to the server deployed at a different domain. As a reference, if the frontend and backend are at two different domains, we need CORS there.

We can setup CORS environment in frontend using two methods:

- axios
- fetch

54. What is axios and how to use it in React?

[Axios](#), which is a popular library is mainly used to send asynchronous HTTP requests to REST endpoints. This library is very useful to perform CRUD operations.

- This popular library is used to communicate with the backend. Axios supports the Promise API, native to JS ES6.
- Using Axios we make API requests in our application. Once the request is made we get the data in Return, and then we use this data in our project.

To install axios package in react use the following command.

```
npm i axios
```

55. Write a program to create a counter with increment and decrement?

We can create the counter app with increment and decrement by writing the below code in the terminal:

```
import React, { useState } from "react";

const App = () => {
  const [counter, setCounter] = useState(0)

  const handleClick1 = () => {
    setCounter(counter + 1)
  }

  const handleClick2 = () => {
    setCounter(counter - 1)
  }

  return (
    <div>
      <div>
        {counter}
```

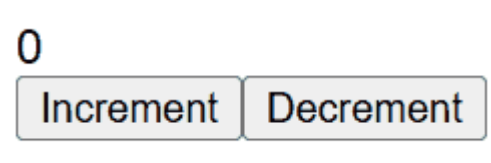
```

    </div>
    <div className="buttons">
      <button onClick={handleClick1}>
        Increment
      </button>
      <button onClick={handleClick2}>
        Decrement
      </button>
    </div>
  </div>
)
}

```

export default App

Output:



56. Explain why and how to update state of components using callback?

It is advised to use a callback-based approach to update the state using `setState` because it solves lots of bugs upfront that may occur in the future. We can use the following syntax to update state using callback

```

this.setState(st => {
  return(
    st.stateName1 = state1UpdatedValue,
    st.stateName2 = state2UpdatedValue
  )
})

```

57. What is React-Material UI?

[React Material UI](#) is an open-source React component library, offering prebuilt components for creating React applications. Developed by Google in 2014, it's compatible with JavaScript frameworks like Angular.js and Vue.js. Renowned for its quality designs and easy customization, it's favored by developers for rapid development.

58. What is flux architecture in redux?

[Flux architecture](#) in Redux is a design pattern used for managing application state in a unidirectional data flow. In this architecture, actions are dispatched to modify the store, which holds the entire application state. The store sends the updated state to the view (UI), and the cycle repeats when new actions are triggered. Redux follows this structure to ensure a predictable and maintainable state management system for large applications.

59. What are custom hooks in React?

[Custom hooks](#) are user-defined functions that use built-in hooks like `useState`, `useEffect`, etc., to reuse stateful logic across components. They allow you to extract and share common logic.

60. How can you optimize React performance?

Optimizing React performance can be achieved using:

- `React.memo()` for preventing unnecessary re-renders.
- Lazy loading components with `React.lazy()` and `Suspense`.
- Using `useMemo()` and `useCallback()` hooks to memoize values and functions.
- Avoiding unnecessary state updates.

61. What is the Strict Mode in React?

Strict Mode in React is a tool that helps developers find and fix problems in their app while developing. It only works in development and doesn't affect the app when it's running in production. When enabled, it checks for potential issues, such as old features that should no longer be used, and gives warnings to help avoid bugs.

We can enable the strict mode by wrapping your application or specific components inside the `<React.StrictMode>`.

```

import React from "react";
import ReactDOM from "react-dom";

const App = () => {
  return (
    <div>
      <h1>Hello, World!</h1>
    </div>
  );
};

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);

```

62. What is Redux and how does it work with React?

Redux is a state management library that helps manage the application state globally. It uses actions, reducers, and a central store to control state. [React-Redux](#) is used to connect Redux state to React components.

63. How does React handle concurrency?

React's Concurrent Mode is a set of features that help React apps stay responsive and gracefully adjust to the user's device capabilities and network speed. It allows React to interrupt rendering and prioritize high-priority tasks.

64. How does React handle server-side rendering (SSR)?

Server-side rendering (SSR) is the process of rendering a React application on the server and sending the fully rendered HTML to the client. This improves the initial page load performance and SEO.

65. What are forms in React?

In React, **forms** are a way to capture user input, such as text fields, checkboxes, radio buttons, and buttons. React provides controlled and uncontrolled components for handling form elements, allowing developers to manage user input and form state in an efficient and structured way.

66. How to create forms in React?

Creating forms in React involves handling user input and managing the form's state. In React, form elements like `<input>`, `<textarea>`, and `<select>` are controlled components, meaning their values are controlled by the React state.

- **Create the State for the Form:** Use the `useState` hook to manage form input values.
- **Set the Value of Form Elements:** Bind the form elements' value attribute to the corresponding state variable to make the inputs controlled.
- **Handle User Input:** Use an `onChange` event handler to update the state whenever the user types in the form fields.
- **Submit the Form:** Handle the form submission with an `onSubmit` event, and prevent the default behavior to stop the page from refreshing.

```
import React, { useState } from 'react';
```

```
function MyForm() {
```

```
  // State to store input values
```

```
  const [name, setName] = useState("");
```

```
  const [email, setEmail] = useState("");
```

```
  const [message, setMessage] = useState("");
```

```
  // Handle input change for each field
```

```
  const handleNameChange = (e) => setName(e.target.value);
```

```
  const handleEmailChange = (e) => setEmail(e.target.value);
```

```
  const handleMessageChange = (e) => setMessage(e.target.value);
```

```
  // Handle form submission
```

```
  const handleSubmit = (e) => {
```

```
    e.preventDefault(); // Prevent default form submission behavior
```

```

    console.log('Form submitted:', { name, email, message });
  };
  return (
    <form onSubmit={handleSubmit}>
      <div>
        <label>Name:</label>
        <input type="text" value={name} onChange={handleNameChange}
      />
      </div>
      <div>
        <label>Email:</label>
        <input type="email" value={email}
      onChange={handleEmailChange} />
      </div>
      <div>
        <label>Message:</label>
        <textarea value={message} onChange={handleMessageChange} />
      </div>
      <button type="submit">Submit</button>
    </form>
  );
}
export default MyForm;

```

Output:

Name:

Email:

Message:

67. What is Lazy Loading in React?

[Lazy Loading](#) in React is a technique used to load components only when they are needed, instead of loading everything at once when the app starts. This helps improve the performance of the app by reducing the initial loading time.

In React, `React.lazy()` is used to implement lazy loading for components, which allows you to split your code into smaller bundles and load them only when required.

68. How is React different from React Native?

React	React Native
A JavaScript library for building user interfaces, primarily for web applications.	A framework for building mobile applications using JavaScript and React.
Focuses on developing web applications.	Focuses on developing native mobile applications for iOS and Android.
Renders HTML using the Virtual DOM .	Renders native mobile components using native APIs (e.g., View, Text, Image).
Uses web technologies like HTML, CSS , and JavaScript.	Uses native mobile components and styles for building mobile UIs.
Runs in the browser on a web server.	Runs on mobile devices and communicates with native code.
Can be deployed on browsers like Chrome, Firefox, etc.	Can be deployed as native apps on iOS and Android platforms.

69. What is Memoization in React?

Memoization in React is an optimization technique used to improve the performance of a component by preventing unnecessary re-renders. It involves caching the results of expensive function calls and reusing the cached result when the inputs to the function haven't changed. This is particularly useful when a component's rendering process is slow, and you want to avoid recalculating or re-rendering unnecessarily.

How Memoization Works in React:

React provides two key APIs for memoization:

1. **React.memo()**: This is a higher-order component (HOC) used to prevent re-renders of functional components if their props have not changed.
2. **useMemo()**: A hook that memorizes the result of a computation and only recomputes it when its dependencies change.

70. What is Reconciliation in React?

Reconciliation in React is the process of updating the **DOM** when a component's state or props change. React uses the **Virtual DOM** to efficiently determine what parts of the actual DOM need to be updated. Here's a simplified overview:

- **Triggering Reconciliation**: React triggers reconciliation when state or props change (e.g., through `setState()` or `useState()`).
- **Virtual DOM Diffing**: React compares the old Virtual DOM with the new one to identify changes.
- **Efficient Updates**: React calculates the minimal set of changes and applies them to the real DOM.
- **Fiber Algorithm**: React's Fiber algorithm allows the reconciliation process to be interrupted and prioritized, improving performance for complex tasks.

1. What is conditional rendering in React?

Conditional rendering in React refers to rendering different components or UI elements based on specific conditions. It allows developers to control what gets displayed in the UI dynamically, depending on the application's state or props. This is similar to using if-else statements in JavaScript but implemented in a declarative way within JSX. Conditional rendering is commonly used for

scenarios like showing a login or logout button based on user authentication status, displaying loading spinners, or toggling UI elements.

Example:

```
function UserStatus({ isLoggedIn }) {  
  return (  
    <div>  
      {isLoggedIn ? <DisplayLoggedIn /> : <DisplayLoggedOut />}  
    </div>  
  );  
}  
function DisplayLoggedIn() {  
  return <h1>Welcome, User!</h1>;  
}  
function DisplayLoggedOut() {  
  return <h1>Please Log In</h1>;  
}
```

In this example, if `isLoggedIn` is true, the `DisplayLoggedIn` component is rendered; otherwise, `DisplayLoggedOut` is shown. You can also use logical operators (`&&`, `||`) or ternary operators for concise conditional rendering.

Key Points:

- Use ternary operators (`?:`) for simple conditions within JSX.
- Use `&&` for rendering elements only when a condition is true (e.g., `{isLoading && <Spinner />}`).
- Avoid complex logic in JSX; move it to a separate function for readability.
- Conditional rendering enhances UI flexibility and user experience.

2. What is React Router?

[React Router](#) is a standard library for handling navigation and routing in React applications. It allows developers to create single-page applications (SPAs) with multiple views, enabling seamless navigation without full page reloads. React Router synchronizes the UI with the browser's URL, ensuring the displayed content matches the current route. It's essential for building dynamic, multi-page-like experiences in SPAs.

Installation:

To use React Router, install it via npm:

```
npm install react-router-dom
```

Example:

```
import { BrowserRouter as Router, Route, Switch, Link } from 'react-router-dom';  
function App() {  
  return (  
    <Router>  
      <nav>  
        <Link to="/">Home</Link>  
        <Link to="/about">About</Link>  
      </nav>  
      <Switch>  
        <Route exact path="/" component={Home} />  
        <Route path="/about" component={About} />  
      </Switch>  
    </Router>  
  );  
}
```

Here, Link creates navigable links, Route maps URLs to components, and Switch ensures only the first matching route is rendered.

Key Points:

- React Router enables client-side routing, improving performance.
- It supports dynamic routing, nested routes, and route parameters.
- Use react-router-dom for web applications (as opposed to react-router-native for mobile apps).

3. Explain the components of React Router.

React Router provides several core components to manage routing in a React application. These components work together to define routes, handle navigation, and render appropriate UI based on the URL.

Main Components:

1. **BrowserRouter (Router):** The parent component that wraps the entire application to enable routing. It uses the HTML5 history API to keep the UI in sync with the URL.

```
import { BrowserRouter as Router } from 'react-router-dom';  
function App() {  
  return <Router>{/* Other components */}</Router>;  
}
```

2. **Switch:** Renders the first Route or Redirect that matches the current URL. It prevents multiple routes from rendering simultaneously.

```
<Switch>  
  <Route path="/home" component={Home} />  
  <Route path="/about" component={About} />  
</Switch>
```

3. **Route:** Defines a mapping between a URL path and a component. It renders the specified component when the URL matches the path. The exact prop ensures precise matching.

```
<Route exact path="/home" component={Home} />
```

4. **Link:** Creates clickable links for navigation without reloading the page. It replaces traditional <a> tags to work with React Router.

```
<Link to="/about">Go to About</Link>
```

Key Points:

- Use BrowserRouter for web apps and HashRouter for static sites.
- Switch ensures only one route renders at a time, improving performance.
- Route supports dynamic parameters (e.g., /user/:id) and query strings.
- Link enhances user experience by enabling smooth navigation.

4. Explain the lifecycle methods of components.

In React, [component lifecycle methods](#) are special methods that allow developers to hook into specific phases of a component's life, such as creation, updating, and removal. These methods are primarily used in class-based components and are divided into four phases:

1. **Initialization:** The component is created with initial props and state, typically in the constructor.

```
class MyComponent extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { count: 0 };  
  }  
}
```

2. **Mounting:** The component is rendered to the DOM for the first time. Key methods include `componentWillMount` (deprecated) and `componentDidMount`.
3. **Updating:** The component re-renders due to changes in props or state. Methods like `shouldComponentUpdate` and `componentDidUpdate` are used here.
4. **Unmounting:** The component is removed from the DOM, using `componentWillUnmount` for cleanup tasks.

Example:

```
class MyComponent extends React.Component {  
  componentDidMount() {  
    console.log('Component mounted!');  
  }  
  componentDidUpdate() {  
    console.log('Component updated!');  
  }  
  componentWillUnmount() {  
    console.log('Component will unmount!');  
  }  
  render() {  
    return <div>Hello, World!</div>;  
  }  
}
```

Key Points:

- Lifecycle methods are less common with functional components, where hooks like `useEffect` replace them.

- Use lifecycle methods for tasks like data fetching, subscriptions, or cleanup.
- Avoid deprecated methods like `componentWillMount` in modern React.

5. Explain the methods used in the mounting phase of components.

The mounting phase occurs when a component is initialized and added to the DOM. During this phase, React calls specific lifecycle methods to allow developers to perform setup tasks. The two primary methods in this phase are:

1. **`componentWillMount()`** (Deprecated): Called just before the component is mounted to the DOM. It was used for setup tasks but is now replaced by `componentDidMount` or hooks due to potential issues in server-side rendering.
2. **`componentDidMount()`**: Called immediately after the component is mounted to the DOM. It's ideal for tasks like fetching data, setting up event listeners, or initializing third-party libraries.

Example:

```
class MyComponent extends React.Component {
  componentDidMount() {
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(data => this.setState({ data }));
  }
  render() {
    return <div>Data: {this.state.data}</div>;
  }
}
```

Key Points:

- Use `componentDidMount` for side effects like API calls or DOM manipulation.
- Avoid using `componentWillMount` in new code; use `componentDidMount` or `useEffect` instead.
- Mounting methods run only once during a component's lifecycle.

6. What is the `this.setState` function in React?

The [setState method](#) in React is used to update a component's state object, triggering a re-render to reflect the updated state in the UI. It's a core feature of class-based components, ensuring that state changes are predictable and efficient. Unlike directly modifying `this.state`, `setState` notifies React to update the component and its children.

Example:

```
class Counter extends React.Component {
  state = { count: 0 };
  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };
  render() {
    return (
      <div>
        <p>Count: {this.state.count}</p>
        <button onClick={this.increment}>Increment</button>
      </div>
    );
  }
}
```

Key Points:

- `setState` is asynchronous, so don't rely on immediate state updates. Use a callback if needed:

```
this.setState({ count: this.state.count + 1 }, () => console.log('State updated'));
```

- You can pass a function to `setState` for updates based on the previous state:

```
this.setState(prevState => ({ count: prevState.count + 1 }));
```

- `setState` merges the provided object with the current state, preserving unmentioned properties.

7. What is the use of `ref` in React?

[Refs](#) in React provide a way to directly access DOM elements or React components created in the render method. They are useful for tasks like focusing an input, triggering animations, or integrating with third-party

libraries. Refs are created using `React.createRef()` or the `useRef` hook in functional components.

Example (Class Component):

```
class InputField extends React.Component {
  constructor(props) {
    super(props);
    this.inputRef = React.createRef();
  }
  focusInput = () => {
    this.inputRef.current.focus();
  };
  render() {
    return (
      <div>
        <input ref={this.inputRef} />
        <button onClick={this.focusInput}>Focus Input</button>
      </div>
    );
  }
}
```

Example (Functional Component):

```
import { useRef } from 'react';
function InputField() {
  const inputRef = useRef(null);
  const focusInput = () => {
    inputRef.current.focus();
  };
  return (
    <div>
      <input ref={inputRef} />
      <button onClick={focusInput}>Focus Input</button>
    </div>
  );
}
```

Key Points:

- Use refs sparingly; prefer props and state for most interactions.
- Refs can store mutable values that persist across renders (via useRef).
- Avoid using refs for tasks that can be handled declaratively with React's state or props.

8. What are hooks in React?

[Hooks](#) are functions introduced in React 16.8 that allow functional components to use state, lifecycle methods, and other React features without writing class components. Hooks simplify code, improve reusability, and align with React's functional programming paradigm. They don't violate React's core concepts but provide a cleaner API for state, context, refs, and more.

Common Hooks:

- `useState`: Manages state in functional components.
- `useEffect`: Handles side effects like data fetching or subscriptions.
- `useContext`: Accesses context for global state.
- `useRef`: Creates references to DOM elements or persistent values.

Example:

```
import { useState, useEffect } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

Key Points:

- Hooks can only be used in functional components or custom hooks.

- They must be called at the top level of a component (not inside loops or conditions).
- Custom hooks allow reusable logic across components.

9. Explain the useState hook in React?

The [useState hook](#) allows functional components to manage state. It returns a state variable and a function to update it, enabling dynamic UI updates. Each call to useState creates a single state variable, stored in React's internal state management, and triggers a re-render when updated.

Syntax:

```
import { useState } from 'react';
const [state, setState] = useState(initialValue);
```

Example:

```
import { useState } from 'react';
function TextInput() {
  const [text, setText] = useState("");
  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <p>You typed: {text}</p>
    </div>
  );
}
```

Key Points:

- useState can store any data type (strings, numbers, objects, arrays).
- Updates via setState are asynchronous; use a function for updates based on the previous state:

```
setCount(prevCount => prevCount + 1);
```

- Multiple useState calls can manage different state variables in a single component.

10. Explain the useEffect hook in React?

The [useEffect hook](#) handles side effects in functional components, such as data fetching, subscriptions, or DOM manipulation. It replaces lifecycle methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`. `useEffect` runs after every render unless controlled by a dependency array.

Syntax:

```
import { useEffect } from 'react';
useEffect(() => {
  // Side effect logic
  return () => {
    // Cleanup logic
  };
}, [dependencies]);
```

Example:

```
import { useState, useEffect } from 'react';
function DataFetcher() {
  const [data, setData] = useState(null);
  useEffect(() => {
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(data => setData(data));
    return () => {
      console.log('Cleanup');
    };
  }, []); // Empty array means run once on mount
  return <div>{data ? data.name : 'Loading...'}</div>;
}
```

Key Points:

- The optional dependency array controls when `useEffect` runs (e.g., `[]` for mount only, `[value]` for changes to value).
- The cleanup function (returned from `useEffect`) runs before the next effect or on unmount.
- Use multiple `useEffect` calls to separate unrelated side effects for clarity.

11. What is React Fragments?

[React Fragments](#) allow developers to group multiple elements without adding extra DOM nodes (like a `<div>`). They solve the issue of JSX requiring a single parent element while keeping the DOM clean. Introduced in React 16.2, Fragments improve performance and semantics by avoiding unnecessary wrapper elements.

Syntax:

```
<React.Fragment>
  <h2>Child 1</h2>
  <p>Child 2</p>
</React.Fragment>
// Shorthand syntax
<>
  <h2>Child 1</h2>
  <p>Child 2</p>
</>
```

Example:

```
function ListItems() {
  return (
    <>
      <li>Item 1</li>
      <li>Item 2</li>
    </>
  );
}
```

Key Points:

- Use Fragments to avoid extra `<div>` tags in the DOM, improving performance.
- The shorthand `<></>` is commonly used but doesn't support keys or attributes.
- Fragments are ideal for rendering lists or grouped elements cleanly.

12. What is a React Developer Tool?

[React Developer Tools](#) is a browser extension for Chrome and Firefox that aids in debugging React applications. It integrates with browser DevTools, allowing developers to inspect the component hierarchy, view props, state, hooks, and performance metrics. It's invaluable for troubleshooting and optimizing React apps.

Features:

- Inspect component trees and their props/state.
- Debug hooks like useState and useEffect.
- Profile performance to identify slow renders.

Installation:

Install from the Chrome Web Store or Firefox Add-ons. Once added, it appears as a “React” tab in DevTools.

Example Use Case:

When debugging, select a component in the React tab to view its props, state, and hooks. You can also edit state in real-time to test UI changes.

Key Points:

- Essential for diagnosing rendering issues or state mismanagement.
- Works with both development and production builds (though limited in production).
- Use alongside browser DevTools for a complete debugging experience.

13. How to use styles in ReactJS?

Styling in React can be done in multiple ways, but [CSS Modules](#) are a popular approach for locally scoped styles. CSS Modules allow you to write CSS files with unique, auto-generated class names to avoid naming conflicts, improving modularity and maintainability.

Steps to Use CSS Modules:

1. Create a CSS file named [ComponentName].module.css (e.g., App.module.css).
2. Write CSS with standard class names.
3. Import and use the styles in your component.

Example (App.module.css):

```
.container {  
  background-color: lightblue;  
  padding: 20px;  
}
```

Example (App.js):

```
import styles from './App.module.css';  
function App() {  
  return <div className={styles.container}>Styled Component</div>;  
}
```

Key Points:

- CSS Modules scope styles locally, preventing global conflicts.
- Use camelCase for class names in JavaScript (e.g., styles.myClass).
- Other styling options include inline CSS, styled-components, or CSS-in-JS libraries.

14. Explain styled-components in React.

[Styled-components](#) is a CSS-in-JS library that allows developers to write CSS within JavaScript, creating reusable, component-scoped styles. It eliminates the need for separate CSS files and provides dynamic styling based on props or state, enhancing developer experience and maintainability.

Installation:

```
npm install styled-components
```

Example:

```
import styled from 'styled-components';  
  
const Button = styled.button`  
  background-color: ${props => (props.primary ? 'blue' : 'gray')};  
  color: white;  
  padding: 10px 20px;  
  border: none;  
  cursor: pointer;
```

```
`;  
function App() {  
  return (  
    <div>  
      <Button>Normal Button</Button>  
      <Button primary>Primary Button</Button>  
    </div>  
  );  
}
```

Key Points:

- Styles are scoped to components, avoiding global CSS conflicts.
- Supports dynamic styling with props and theming.
- Integrates well with TypeScript and modern React workflows.

15. What is prop drilling and its disadvantages?

[Prop drilling](#) occurs when data is passed from a parent component to deeply nested child components through props, even if intermediate components don't need the data. While functional, this practice can lead to cumbersome and hard-to-maintain code.

Example:

```
function App() {  
  const user = { name: 'John' };  
  return <Parent user={user} />;  
}  
function Parent({ user }) {  
  return <Child user={user} />;  
}  
function Child({ user }) {  
  return <GrandChild user={user} />;  
}  
function GrandChild({ user }) {  
  return <div>{user.name}</div>;  
}
```

Disadvantages:

- **Code Clutter:** Intermediate components must pass props they don't use, increasing complexity.
- **Maintenance Issues:** Refactoring or adding new props requires changes across multiple components.
- **Readability:** Long prop chains make code harder to follow.

Solutions:

- Use [Context API](#) to share data globally.
- Use state management libraries like Redux or Zustand for complex apps.

16. What are controlled and uncontrolled components in React?

[Controlled](#) and [uncontrolled components](#) refer to how form elements (like inputs) manage their data in React.

- **Controlled Components:** The parent component manages the form element's value via state. The input's value is bound to a state variable, and changes are handled through event handlers.
- **Uncontrolled Components:** The form element maintains its own internal state, and React accesses its value using refs.

Example (Controlled Component):

```
function ControlledInput() {
  const [value, setValue] = useState("");
  return (
    <input
      value={value}
      onChange={(e) => setValue(e.target.value)}
    />
  );
}
```

Example (Uncontrolled Component):

```
function UncontrolledInput() {
  const inputRef = useRef(null);
  const handleSubmit = () => {
    alert(inputRef.current.value);
  };
}
```

```

};
return (
  <div>
    <input ref={inputRef} />
    <button onClick={handleSubmit}>Submit</button>
  </div>
);
}

```

Key Points:

- Controlled components are preferred for predictable data flow and validation.
- Uncontrolled components are simpler for basic forms but harder to manage in complex scenarios.
- Use controlled components for forms requiring real-time validation or synchronization.

17. What is the useRef hook in React?

The [useRef hook](#) creates a mutable reference that persists across renders in functional components. It's commonly used to access DOM elements or store values that don't trigger re-renders when updated. The useRef object has a .current property that holds the reference's value.

Example:

```

import { useRef, useEffect } from 'react';
function AutoFocusInput() {
  const inputRef = useRef(null);
  useEffect(() => {
    inputRef.current.focus();
  }, []);
  return <input ref={inputRef} />;
}

```

Key Points:

- useRef doesn't cause re-renders when .current changes.

- Useful for accessing DOM nodes, storing previous values, or managing timers.
- Unlike state, useRef values persist for the component's entire lifecycle.

18. Explain the componentDidMount method in React.

Explanation:

The [componentDidMount method](#) is a lifecycle method in class-based components that runs immediately after a component is mounted to the DOM. It's ideal for initializing tasks like fetching data, setting up subscriptions, or manipulating the DOM.

Example:

```
class DataFetcher extends React.Component {
  state = { data: null };
  componentDidMount() {
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(data => this.setState({ data }));
  }
  render() {
    return <div>{this.state.data ? this.state.data.name : 'Loading...'}</div>;
  }
}
```

Key Points:

- Runs only once after the initial render.
- Equivalent to useEffect with an empty dependency array in functional components.
- Avoid setting state in a loop to prevent performance issues.

19. Difference between ES6 and ES5 syntax in React.

[ES6](#) (ECMAScript 2015) introduced modern JavaScript features that simplify React development compared to [ES5](#). These differences impact how components, imports, and functions are written.

Comparison Table:

Feature	ES5	ES6
Importing React	<code>var React = require('react');</code>	<code>import React from 'react';</code>
Exporting Components	<code>module.exports = Component;</code>	<code>export default Component;</code>
Function Declaration	<code>var sum = function(x, y) { return x + y; };</code>	<code>const sum = (x, y) => x + y;</code>
Class Components	<code>var MyComponent = React.createClass({ ... });</code>	<code>class MyComponent extends React.Component { ... }</code>
State Initialization	<code>this.state = { count: 0 }; (in constructor)</code>	<code>state = { count: 0 }; (Class Fields)</code>
Method Binding	<code>this.handleClick = this.handleClick.bind(this); (in constructor)</code>	No binding needed with arrow functions: <code>handleClick = () => { ... };</code>

Example (ES5):

```
var MyComponent = React.createClass({
  getInitialState: function() {
    return { count: 0 };
  },
  handleClick: function() {
    this.setState({ count: this.state.count + 1 });
  },
  render: function() {
    return <button
```

```
onClick={this.handleClick.bind(this)}>{this.state.count}</button>;  
  }  
});
```

Example (ES6):

```
import React from 'react';  
class MyComponent extends React.Component {  
  state = { count: 0 };  
  handleClick = () => {  
    this.setState({ count: this.state.count + 1 });  
  };  
  render() {  
    return <button onClick={this.handleClick}>{this.state.count}</button>;  
  }  
}  
export default MyComponent;
```

Key Points:

- ES6 syntax is more concise and readable, especially with arrow functions and class fields.
- ES5 requires manual binding of methods, while ES6 arrow functions eliminate this need.
- Modern React projects use ES6+ for better developer experience and compatibility.

20. What are synthetic events in React?

[Synthetic events](#) are React's cross-browser wrapper around native browser events. They normalize event properties and behavior to ensure consistency across different browsers. Synthetic events are used in event handlers (e.g., `onClick`, `onChange`) and provide a unified API for handling user interactions.

Example:

```
function Button() {  
  const handleClick = (event) => {  
    console.log(event.type); // 'click'  
    console.log(event.target); // DOM element
```

```
};
return <button onClick={handleClick}>Click Me</button>;
}
```

Key Points:

- Synthetic events are pooled for performance, so their properties can't be accessed asynchronously.
- They support all standard browser events but are optimized for React's rendering model.
- Use `event.preventDefault()` to stop default browser behavior (e.g., form submission).

21. What is the Context API in React?

The [Context API](#) allows sharing global data (e.g., themes, user authentication) across a React component tree without prop drilling. It provides a Provider to supply data and a Consumer (or `useContext` hook) to access it in nested components.

Syntax:

```
const MyContext = React.createContext(defaultValue);
```

Example:

```
import { createContext, useContext } from 'react';
const ThemeContext = createContext('light');
function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Child />
    </ThemeContext.Provider>
  );
}
```

```
function Child() {
  const theme = useContext(ThemeContext);
  return <div>Current theme: {theme}</div>;
}
```

Key Points:

- Use Context for global data like themes, locales, or auth states.
- Avoid overuse; it can make components harder to test and reuse.
- The useContext hook simplifies consuming context in functional components.

22. How can you optimize the performance of a React application?

Optimizing a React application improves rendering speed and user experience. Key techniques include minimizing re-renders, reducing bundle size, and efficient state management.

Techniques:

1. **Use React.memo:** Prevents unnecessary re-renders of functional components when props don't change.

```
const MyComponent = React.memo(({ data }) => <div>{data}</div>);
```

2. **Lazy Loading and Code Splitting:** Use React.lazy and Suspense to load components only when needed.

```
const LazyComponent = React.lazy(() => import('./LazyComponent'));  
function App() {  
  return (  
    <Suspense fallback={<div>Loading...</div>}>  
      <LazyComponent />  
    </Suspense>  
  );  
}
```

3. **Use useCallback and useMemo:** Cache functions and values to avoid unnecessary recalculations.

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

4. **Avoid Unnecessary Re-renders:** Minimize state changes and use keys in lists to optimize rendering.
5. **Use Production Builds:** Ensure the app is built with NODE_ENV=production to enable optimizations.

Key Points:

- Profile performance with React Developer Tools to identify bottlenecks.
- Optimize images and assets to reduce load times.
- Consider server-side rendering (SSR) with [Next.js](#) for faster initial loads.

23. What is the purpose of the useCallback hook in React?

The [useCallback hook](#) returns a memoized version of a function that only changes if its dependencies change. It's used to prevent unnecessary function recreations, which can cause child components to re-render unnecessarily when passed as props.

Syntax:

```
const memoizedCallback = useCallback(() => {
  // Function logic
}, [dependencies]);
```

Example:

```
import { useState, useCallback } from 'react';
function Parent() {
  const [count, setCount] = useState(0);
  const handleClick = useCallback(() => {
    console.log('Button clicked');
  }, []);
  return <Child onClick={handleClick} />;
}
const Child = React.memo(({ onClick }) => {
  console.log('Child rendered');
  return <button onClick={onClick}>Click</button>;
});
```

Key Points:

- Use useCallback for functions passed to memoized components.
- Dependencies must include all variables used in the function.
- Overusing useCallback can add complexity; use only when needed.

24. What are Higher-Order Components (HOCs) in React?

[Higher-Order Components \(HOCs\)](#) are functions that take a component and return a new component with enhanced functionality. HOCs are used for code reuse, adding features like authentication, logging, or data fetching to components.

Example:

```
function withAuth(Component) {  
  return function WrappedComponent(props) {  
    const isAuthenticated = checkAuth(); // Assume some auth logic  
    if (!isAuthenticated) {  
      return <Redirect to="/login" />;  
    }  
    return <Component {...props} />;  
  };  
}  
const ProtectedComponent = withAuth(MyComponent);
```

Key Points:

- HOCs promote reusability but can increase complexity.
- Alternatives include hooks or render props for similar functionality.
- Use meaningful names for HOCs (e.g., withAuth, withData).

25. What is the purpose of the useReducer hook in React?

The [useReducer hook](#) manages complex state logic in functional components. It's an alternative to useState for scenarios involving multiple state transitions or interdependent state updates, similar to Redux's reducer pattern.

Syntax:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Example:

```
import { useReducer } from 'react';  
const initialState = { count: 0 };  
function reducer(state, action) {  
  switch (action.type) {  
    case 'increment':  
      return { count: state.count + 1 };  
  }  
}
```

```

    case 'decrement':
      return { count: state.count - 1 };
    default:
      return state;
  }
}
function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);
  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment'
    })}>Increment</button>
      <button onClick={() => dispatch({ type: 'decrement'
    })}>Decrement</button>
    </div>
  );
}

```

Key Points:

- Use useReducer for complex state logic with multiple actions.
- Dispatch actions to update state predictably.
- Can be combined with Context API for global state management.

26. What is a React Portal?

[React Portals](#) allow rendering a component's children into a DOM node outside the parent component's DOM hierarchy. They are useful for rendering modals, tooltips, or popovers that need to break out of their parent's CSS or DOM constraints.

Syntax:

```
ReactDOM.createPortal(child, container);
```

Example:

```

import ReactDOM from 'react-dom';
function Modal() {
  return ReactDOM.createPortal(

```

```

    <div className="modal">Modal Content</div>,
    document.getElementById('modal-root')
  );
}

```

HTML:

```

<div id="root"></div>
<div id="modal-root"></div>

```

Key Points:

- Portals don't affect React's component tree, only the DOM.
- Useful for handling z-index or overflow issues in modals.
- Ensure the target DOM node exists (e.g., modal-root).

27. Explain the "Virtual DOM" in React.

The [Virtual DOM](#) is an in-memory representation of the real DOM. React uses it to optimize updates by comparing the new Virtual DOM with the previous one (diffing) and applying only the necessary changes to the real DOM. This process, called reconciliation, improves performance by minimizing costly DOM operations.

How It Works:

1. When state or props change, React creates a new Virtual DOM tree.
2. React compares it with the previous tree to identify differences.
3. Only the changed parts are updated in the real DOM.

Example:

```

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

```

When count updates, React updates only the <p> element's text, not the entire DOM.

Key Points:

- The Virtual DOM reduces direct DOM manipulation, improving performance.
- Diffing is fast because it operates on lightweight JavaScript objects.
- Frameworks like [Next.js](#) enhance Virtual DOM with SSR.

28. What are keys in React and why are they important?

[Keys](#) are unique identifiers used in React lists to help React efficiently update and reconcile elements. They ensure React can track which items have changed, been added, or removed, optimizing rendering performance.

Example:

```
function List() {  
  const items = ['Apple', 'Banana', 'Orange'];  
  return (  
    <ul>  
      {items.map((item, index) => (  
        <li key={index}>{item}</li>  
      ))}  
    </ul>  
  );  
}
```

Key Points:

- Use unique, stable keys (e.g., item IDs) instead of indices to avoid issues with list reordering.
- Incorrect keys can lead to unexpected behavior or performance issues.
- Keys are only needed for elements in arrays, not single components.

29. What is the difference between `React.PureComponent` and `React.Component`?

[React.PureComponent](#) is a base class like `React.Component` but includes an automatic `shouldComponentUpdate` implementation that performs a shallow comparison of props and state to prevent unnecessary re-renders.

Comparison:

Feature	React.Component	React.PureComponent
Re-rendering	Re-renders on every state/prop change	Re-renders only if props/state change (shallow)
Performance	Manual optimization needed	Automatic optimization via shallow comparison
Use Case	General-purpose components	Performance-sensitive components

Example (PureComponent):

```
class MyPureComponent extends React.PureComponent {  
  render() {  
    console.log('Rendering');  
    return <div>{this.props.value}</div>;  
  }  
}
```

Key Points:

- Use `PureComponent` for components with simple props/state to avoid manual `shouldComponentUpdate`.
- Shallow comparison may miss deep object changes; use immutable data or `useMemo` for complex cases.
- Functional components with `React.memo` are the modern alternative.

30. What are the differences between Server-Side Rendering (SSR) and Client-Side Rendering (CSR) in React, and when would you use each?

[Server-Side Rendering \(SSR\)](#) and [Client-Side Rendering \(CSR\)](#) are two approaches to rendering React applications.

Comparison:

Feature	SSR	CSR
Rendering Location	Server renders HTML	Browser renders HTML after JS loads
Initial Load Time	Faster (HTML sent pre-rendered)	Slower (waits for JS bundle)
SEO	Better (search engines see full HTML)	Poor (search engines may miss dynamic content)
Interactivity	Delayed until JS hydrates	Immediate after JS loads
Use Case	SEO-heavy sites, static pages	Interactive SPAs, dashboards

Example (SSR with Next.js):

```
// pages/index.js
function HomePage({ data }) {
  return <div>{data.title}</div>;
}
export async function getServerSideProps() {
  const res = await fetch('https://api.example.com/data');
  const data = await res.json();
  return { props: { data } };
}
```

When to Use:

- **SSR:** Use for public-facing sites, blogs, or e-commerce platforms where SEO and fast initial loads are critical (e.g., with [Next.js](#)).
- **CSR:** Use for private dashboards, SPAs, or apps where interactivity is prioritized over SEO.

1. What are custom hooks in React?

[Custom hooks](#) are normal JavaScript functions whose names start with “use”, and they may call other hooks. We use custom hooks to maintain the DRY concept, which is Don't Repeat Yourself. It helps us to write a logic once and use it anywhere in the code. Some of the commonly used custom hooks are:

- **useFetch:** Used for making API requests and handling loading and error states.
- **useLocalStorage:** Stores and retrieves values from localStorage to persist data between sessions.
- **useDebounce:** Used to delay a function execution until after a specified wait time (useful for search input).
- **usePrevious:** Stores the previous value of a state or prop before the component re-renders.
- **useToggle:** A simple hook to toggle a boolean state.

2. How to optimize a React code?

We can improve our [React](#) code by following these practices:

- Using binding functions in constructors
- Eliminating the use of inline attributes as they slow the process of loading
- Avoiding extra tags by using React fragments
- Lazy loading

3. What is the difference between useRef and createRef in React ?

Feature	useRef	createRef
Type	It is a type of hook.	It is a function.

Feature	useRef	createRef
Component Type	Used in functional components.	Used in class components.
Re-render Behavior	It saves its value between re-renders in a functional component.	It creates a new ref for every re-render.
Ref Value	Returns a mutable ref object ({ current: value }).	Returns a new ref object every time.
Use Cases	Storing DOM references, instance variables, and previous state values without re-renders.	Storing DOM references in class components.
Example	<code>const ref = useRef(null);</code>	<code>this.ref = React.createRef();</code>

4. What is React Redux?

[React Redux](#) is a state management tool which makes it easier to pass these states from one component to another irrespective of their position in the component tree and hence prevents the complexity of the application. As the number of components in our application increases it becomes difficult to pass state as props to multiple components. To overcome this situation we use react-redux.

5. What are the benefits of using react-redux?

There are several benefits of using react-redux such as

- It provides centralized state management i.e. a single store for the whole application
- It optimizes performance as it prevents re-rendering of component
- Makes the process of debugging easier

- Since it offers persistent state management therefore storing data for a long times becomes easier

6. Explain the core components of react-redux.

There are four fundamental concepts of redux in react which decide how the data will flow through components

- **Redux Store:** It is an object that holds the application state
- **Action Creators:** These are unctons that return actions (objects)
- **Actions:** Actions are simple objects which conventionally have two properties- type and payload
- **Reducers:** Reducers are pure functions that update the state of the application in response to actions

7. How can we combine multiple reducers in React?

When working with Redux we sometimes require multiple reducers. In many cases, multiple actions are needed, resulting in the requirement of multiple reducers. However, this can become problematic when creating the Redux store. To manage the multiple reducers we have function called combineReducers in the redux. This basically helps to combine multiple reducers into a single unit and use them.

Syntax

```
import { combineReducers } from "redux";

const rootReducer = combineReducers({
  books: BooksReducer,
  activeBook: ActiveBook
});
```

8. What is context API?

[Context API](#) is used to pass global variables anywhere in the code. It helps when there is a need for sharing state between a lot of nested components. It is light in weight and easier to use, to create a context just need to call `React.createContext()`. It eliminates the need to install other dependencies or

third-party libraries like redux for state management. It has two properties Provider and Consumer.

9. Explain provider and consumer in ContextAPI?

A provider is used to provide context to the whole application whereas a consumer consumes the context provided by nearest provider. In other words The Provider acts as a parent it passes the state to its children whereas the Consumer uses the state that has been passed.

10. Explain CORS in React.Axios

In ReactJS, [Cross-Origin Resource Sharing](#) (CORS) refers to the method that allows you to make requests to the server deployed at a different domain. As a reference, if the frontend and backend are at two different domains, we need CORS there.

We can setup CORS environment in frontend using two methods:

- Axios
- fetch

11. What is axios and how to use it in React?

Axios, which is a popular library is mainly used to send asynchronous HTTP requests to REST endpoints. This library is very useful to perform CRUD operations.

- This popular library is used to communicate with the backend. Axios supports the Promise API, native to JS ES6.
- Using Axios we make API requests in our application. Once the request is made we get the data in Return, and then we use this data in our project.

To install axios package in react use the following command.

```
npm i axios
```

12. Write a program to create a counter with increment and decrement.

```
import React, { useState } from "react";  
  
const App = () => {  
  const [counter, setCounter] = useState(0)
```

```
const handleClick1 = () => {
  setCounter(counter + 1)
}

const handleClick2 = () => {
  setCounter(counter - 1)
}

return (
  <div>
    <div>
      {counter}
    </div>
    <div className="buttons">
      <button onClick={handleClick1}>
        Increment
      </button>
      <button onClick={handleClick2}>
        Decrement
      </button>
    </div>
  </div>
)
}
```

export default App

Output:

0

Increment

Decrement

In this example

- The useState hook initializes a state variable counter with a value of 0. The setCounter function is used to update this state.
- When the Increment button is clicked, setCounter(counter + 1) increases the counter value by 1.
- When the Decrement button is clicked, setCounter(counter - 1) decreases the counter value by 1.
- The current value of counter is displayed inside a <div> and updates automatically when the state changes.
- Two buttons trigger handleClick1 and handleClick2 using onClick, allowing users to increase or decrease the counter.

13. Explain why and how to update state of components using callback?

It is advised to use a callback-based approach to update the state using setState because it solves lots of bugs upfront that may occur in the future. We can use the following syntax to update the state using a callback.

```
this.setState(st => {  
  return(  
    st.stateName1 = state1UpdatedValue,  
    st.stateName2 = state2UpdatedValue  
  )  
})
```

14. What is React-Material UI?

[React Material UI](#) is a framework built upon React library which contains predefined components to create React applications. Material UI is basically a design language built by Google in 2014 and works with various JavaScript frameworks apart from React such as Angular.js and Vue.js. The quality of the inbuilt designs of Material UI and its easy implementation make it the first choice of most developers. The inbuilt components are also customizable so it helps easily recreate the designs.

15. What is flux architecture in redux?

Flux is an architecture that Facebook uses internally when operating with React. It is merely a replacement quite an architecture that enhances React and also the idea of unidirectional data flow.

16. Explain the useMemo hook in react?

The useMemo is a hook used in the functional component of react that returns a memoized value. In Computer Science, memoization is a concept used in general when we don't need to recompute the function with a given argument for the next time as it returns the cached result. A memoized function remembers the results of output for a given set of inputs.

17. Does React useState Hook update immediately?

React do not update immediately, although it seems immediate at first glance. React keep track of the states by queuing them in the order they are called. React queue all the changes to be made and update once the component Re-render which is not immediate. This is how React knows which value corresponds to which state. It goes as per the queue every time the component tries to re-render.

18. When to use useCallback, useMemo and useEffect ?

- useEffect is a function that can be used as an alternative to lifecycle methods such as componentDidMount, componenetWillUnmount, and componentDidUpdate in functional components
- useCallback can be used when we want to prevent unnecessary renders from the chld components. It helps to resolve side effects
- useMemo is used when we want to re-render based on cache values as makes the application faster

19. Explain the types of router in React?

There are three types of routers in React:

- **Memory Router:** The memory router keeps the URL changes in memory not in the user browsers.
- **Browser Router:** It uses HTML5 history API (i.e. pushState, replaceState, and popState API) to keep your UI in sync with the URL

- **Hash Router:** Hash router uses client-side hash routing. It uses the hash portion of the URL (i.e. `window.location.hash`) to keep your UI in sync with the URL.

20. What is StrictMode in React?

The React StrictMode can be viewed as a helper component that allows developers to code efficiently and brings to their attention any suspicious code which might have been accidentally added to the application. The StrictMode can be applied to any section of the application, not necessarily to the entire application.

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

21. What is Conditional Rendering in React?

[Conditional rendering in React](#) means dynamically displaying components or elements based on a condition. Instead of rendering everything at once, React decides what to render based on the current state, props, or logic. We can use conditional rendering when:

- We have to Show/hide UI elements based on user authentication.
- Display different layouts based on user roles.
- Render UI dynamically based on API responses.
- Show loading spinners while fetching data.

22. What is React Router?

[React Router](#) is a library for handling navigation in React applications. It allows developers to create single-page applications (SPAs) with multiple views while maintaining a smooth, seamless user experience without full-page reloads.

Let's now create the simple react router.

```
// App.js
```

```
import React from "react";
```

```
import { BrowserRouter as Router, Route, Routes, Link } from "react-router-dom";
```

```
// Components for different pages
```

```
const Home = () => <h2>Home Page</h2>;
```

```
const About = () => <h2>About Page</h2>;
```

```
const App = () => {
```

```
  return (
```

```
    <Router>
```

```
      <nav>
```

```
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
```

```
      </nav>
```

```
      <Routes>
```

```
        <Route path="/" element={<Home />} />
```

```
        <Route path="/about" element={<About />} />
```

```
      </Routes>
```

```
    </Router>
```

```
  );
```

```
};
```

```
export default App;
```

Output

In this example

- The code imports React to create components and ReactDOM to render the app to the browser.
- Instead of ReactDOM.render(), we use ReactDOM.createRoot() to create a "root" where our app will be attached in the DOM.
- document.getElementById("root") grabs the div element with the id="root" in your index.html. This is where the React app will be rendered.
- root.render(<App />) tells React to render the App component inside the root element created earlier.

Note: React 18 introduces this new way of rendering the app. The createRoot() method is now used instead of the old render() method to improve performance and enable new features like concurrent rendering.

23. What is the difference between the React Hooks and Classes?

Feature	React Hooks	Class Components
State Management	useState() hook for state handling.	this.state for state handling.
Reusability	Custom hooks allow sharing logic across components.	Inheritance is used for sharing logic, but it's more complex.
this Keyword	No use of this. State and methods are directly accessible.	Uses this to access state and methods (this.state, this.setState()).
Lifecycle Methods	useEffect() for side effects and lifecycle management.	Methods like componentDidMount(), componentDidUpdate(), etc.

Feature	React Hooks	Class Components
Performance	React's new hooks API has better performance optimizations.	Generally less efficient compared to functional components and hooks.

24. How Data is passed between the React Components?

In React, data is passed between components using props. The way data flows between components depends on whether they are parent-child, sibling components, or if you're using state management techniques like context or Redux.

25. What is Lazy Loading in React?

[Lazy Loading](#) in React is a technique used to load components only when they are needed, instead of loading everything at once when the app starts. This helps improve the performance of the app by reducing the initial loading time. In React, `React.lazy()` is used to implement lazy loading for components, which allows you to split your code into smaller bundles and load them only when required.

26. Explain Redux and its components.

Redux is a state management library that helps manage the application state globally. [React-Redux](#) is used to connect Redux state to React components. There are the three main components of the redux:

- **store:** It holds the application state
- **actions:** It describe what to do
- **reducers:** It define how the state changes in response to actions

27. How React Routing is different from Conventional Routing?

React Routing is used in [Single Page Applications \(SPAs\)](#), while conventional routing typically applies to multi-page applications. Here's how they differ:

Features	React Routing	Conventional Routing
Page Loading	No full page reload; components are dynamically rendered.	Entire page reload happens with every navigation.
SEO Considerations	React Router needs additional configurations (like SSR or dynamic rendering) to optimize SEO.	SEO is typically easier to manage since pages are fully rendered by the server.
Navigation	Navigation is handled by JavaScript on the client-side, often using Link and NavLink components.	Navigation is handled by the server through HTML anchor tags ().
Technology	Managed via React Router; uses client-side routing.	Managed by server-side routing (e.g., Express, PHP).
URL Structure	URLs can have dynamic parameters handled by React Router (e.g., /product/:id).	URLs are typically static and are managed by the server.

28. How to avoid binding in ReactJS?

In ReactJS, binding in the context of event handlers is often required to ensure that the correct this context is available inside class methods. However, it can lead to boilerplate code and performance issues, especially when binding functions in each render cycle. We can avoid binding in ReactJS by using the below methods:

- Arrow Functions
- Higher-Order Components (HOCs) or Custom Hooks

29: What are props in React?

In React, props (short for "properties") are a way of passing data from a parent component to a child component. Props are read-only and are used to customize or control the behavior of a component. They help make components reusable and dynamic by allowing different data to be passed into them.

30. What is the use of CSS modules in React?

CSS Modules help you keep your styles specific to each React component, so they don't affect other components. In simple terms, it lets you write CSS for one component without worrying about messing up the styles of other parts of your app. This is especially useful when your app grows larger and has many components, making it easier to manage styles without conflicts.